

Lucas Augusto da Silva Cardoso (202200332300)

DCA3703 | Programação Paralela | Relato de experimento

TAREFA 10: IMPACTO DAS DIRETIVAS DE SINCRONIZAÇÃO E REDUÇÃO OPENMP

Este documento detalha a abordagem metodológica e a solução implementada para o Experimento Computacional proposto na disciplina de Programação Paralela, em 04/09/2026.

1. EXPERIMENTO

Implemente novamente o estimador da tarefa 8 usando um contador compartilhado e o "rand_r" protegendo a soma de hits com "#pragma omp critical" e com "#pragma omp atomic". Compare essas duas implementações com suas versões que usam contadores privados. Agora, compare essas com uma 5ª versão que utiliza apenas a cláusula reduction ao invés das diretivas de sincronização. Reflita sobre a aplicabilidade de desses mecanismos em termos de desempenho e produtividade e proponha um roteiro para quando utilizar qual mecanismo de sincronização, incluindo critical nomeadas e locks explícitos.

2. CONCEITOS RELEVANTES

- **#pragma omp atomic:** com ele, o hardware garante que nenhuma outra thread possa ler ou escrever naquele endereço exato de memória até que o ciclo de "Leitura-Modificação-Escrita" seja totalmente concluído por quem o iniciou (instruções atômicas de hardware).
- **reduction**: é usada para combinar os resultados de operações individuais realizadas por múltiplas threads em uma única variável final, garantindo tanto a correção dos dados quanto o máximo desempenho possível

3. MÁQUINA UTILIZADA

Para a execução deste experimento, estou utilizando uma máquina equipada com um processador i7 de 13ª geração (16 núcleos físicos e 24 threads), que é o componente central para a avaliação das diretivas OpenMP. O sistema dispõe também de 16 GB de memória RAM e um SSD de 512GB.

4. CÓDIGO

Para a realização deste experimento, foram desenvolvidas cinco variações do estimador de Monte Carlo para o cálculo do valor de “pi”, variando as estratégias de sincronização e a granularidade do acesso aos dados. A Figura 1 apresenta a versão inicial, na qual o incremento do contador compartilhado ocorre diretamente dentro do laço principal utilizando “#pragma omp critical”. Como essa diretiva força um bloqueio global de *software*, a execução do laço torna-se praticamente serializada devido ao alto nível de contenção entre as *threads*. Na Figura 2, mitiga-se esse gargalo ao adotar um contador privado (contador_local), acumulando os acertos isoladamente em cada *thread* e aplicando o “#pragma omp critical” apenas uma única vez ao término da região paralela.

Buscando otimizar o acesso atômico no nível de hardware, a Figura 3 substitui a seção crítica por “#pragma omp atomic” dentro do laço de execução. Embora o *overhead* das instruções atômicas seja significativamente menor que o de seções críticas, o acesso concorrente contínuo à mesma posição de memória ainda impõe contenção no barramento de dados. Essa limitação é contornada na “Figura 4”, que combina a abordagem de contadores locais ao “#pragma omp atomic” final, reduzindo drasticamente os acessos à memória compartilhada. Por fim, a Figura 5 utiliza a cláusula de abstração de alto nível “reduction(+:pontos_no_circulo)”. Nessa implementação, o compilador gerencia automaticamente a criação, a inicialização e a agregação eficiente dos contadores privados em árvore, eliminando completamente a necessidade de diretivas explícitas de sincronização no código e oferecendo o melhor compromisso entre desempenho e produtividade.

Figura 1: Versão usando critical dentro do laço

```c
#include <math.h>
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define PI 3.14159265358979323846

int main(int argc, char **argv) {
   long long total_pontos = 10000000LL;

   long long pontos_no_circulo = 0;
   double t_inicio = omp_get_wtime();

   #pragma omp parallel shared(total_pontos, pontos_no_circulo)
   {
      unsigned int seed = (unsigned int)time(NULL) ^ (unsigned int)(omp_get_thread_num()
+ 1);

      #pragma omp for
      for (long long i = 0; i < total_pontos; i++) {
         double x = (double)rand_r(&seed) / (double)RAND_MAX;
         double y = (double)rand_r(&seed) / (double)RAND_MAX;

         if (x * x + y * y <= 1.0) {
            #pragma omp critical
            {
               pontos_no_circulo++;
            }
         }
      }
   }

   double tempo_total = omp_get_wtime() - t_inicio;
   double pi_estimado = 4.0 * (double)pontos_no_circulo / (double)total_pontos;

   printf("Threads ativas : %d\n", omp_get_max_threads());
```

```
    printf("Total de pontos: %lld\n", total_pontos);
    printf("Pontos no circulo : %lld / %lld\n", pontos_no_circulo, total_pontos);
    printf("Pi Estimado       : %.10f\n", pi_estimado);
    printf("Pi Real (PI)      : %.10f\n", PI);
    printf("Erro Absoluto     : %.10e\n", fabs(pi_estimado - PI));
    printf("Tempo de Execucao : %.6f segundos\n", tempo_total);

    return 0;
}
```

Figura 2: Versão usando critical + contador local

```
#define PI 3.14159265358979323846

int main(int argc, char *argv[]) {
    long long total_pontos = 10000000LL; // 10 milhões de pontos

    printf("Threads ativas : %d\n", omp_get_max_threads());
    printf("Total de pontos: %lld\n", total_pontos);

    long long pontos_no_circulo_total = 0; // Compartilhada entre todas as threads
    long long contador_local;              // Será tornada 'private' para cada thread

    double t_inicio = omp_get_wtime();

    // 1. Região paralela: define o que é compartilhado e o que é privado
    #pragma omp parallel \
        shared(total_pontos, pontos_no_circulo_total) \
        private(contador_local)
    {
        int tid = omp_get_thread_num();
        unsigned int seed = (unsigned int)time(NULL) + tid;

        // 'contador_local' é private, portanto inicia com LIXO DE MEMÓRIA.
        // inicializando manualmente aqui dentro:
        contador_local = 0;

        double x, y, dist_sq; // Variáveis locais à thread (privadas)

        // Distribuindo as iterações do loop entre as threads
        #pragma omp for private(x, y, dist_sq)
```

```
        for (long long i = 0; i < total_pontos; i++) {
            x = (double)rand_r(&seed) / (double)RAND_MAX;
            y = (double)rand_r(&seed) / (double)RAND_MAX;
            dist_sq = x * x + y * y;

            if (dist_sq <= 1.0) {
                contador_local++; // Incremento local sem condição de corrida
            }
        }

        // Atualizando o total global apenas uma única vez por thread
        #pragma omp critical
        {
            pontos_no_circulo_total += contador_local;
        }
    }

    double t_fim = omp_get_wtime();
    double tempo_total = t_fim - t_inicio;

    double pi_estimado = 4.0 * ((double)pontos_no_circulo_total / (double)total_pontos);
    double erro_absoluto = fabs(pi_estimado - PI);

    printf("Pontos no circulo : %lld / %lld (Correto!)\n", pontos_no_circulo_total, total_pontos);
    printf("Pi Estimado       : %.10f (Correto!)\n", pi_estimado);
    printf("Pi Real (PI)      : %.10f\n", PI);
    printf("Erro Absoluto     : %.10e\n", erro_absoluto);
    printf("Tempo de Execucao : %.6f segundos\n", tempo_total);

    return 0;
}
```

Figura 3: Versão com atomic dentro do laço

```
#define PI 3.14159265358979323846

int main(int argc, char **argv) {
    long long total_pontos = 10000000LL;
    long long pontos_no_circulo = 0;

    double t_inicio = omp_get_wtime();
```

```
    #pragma omp parallel shared(total_pontos, pontos_no_circulo)
    {
        unsigned int seed = (unsigned int)time(NULL) ^
            (unsigned int)(omp_get_thread_num() + 1);

        #pragma omp for
        for (long long i = 0; i < total_pontos; i++) {
            double x = (double)rand_r(&seed) / (double)RAND_MAX;
            double y = (double)rand_r(&seed) / (double)RAND_MAX;

            if (x * x + y * y <= 1.0) {
                #pragma omp atomic
                pontos_no_circulo++;
            }
        }
    }

    double tempo_total = omp_get_wtime() - t_inicio;
    double pi_estimado = 4.0 * (double)pontos_no_circulo / (double)total_pontos;

    printf("Threads ativas : %d\n", omp_get_max_threads());
    printf("Total de pontos: %lld\n", total_pontos);
    printf("Pontos no circulo : %lld / %lld\n", pontos_no_circulo, total_pontos);
    printf("Pi Estimado       : %.10f\n", pi_estimado);
    printf("Pi Real (PI)      : %.10f\n", PI);
    printf("Erro Absoluto     : %.10e\n", fabs(pi_estimado - PI));
    printf("Tempo de Execucao : %.6f segundos\n", tempo_total);

    return 0;
}
```

Figura 4: Versão com atomic + contador local

```
#define PI 3.14159265358979323846

int main(int argc, char **argv) {
    long long total_pontos = 10000000LL;
    long long pontos_no_circulo = 0;

    double t_inicio = omp_get_wtime();
```

```
    #pragma omp parallel shared(total_pontos, pontos_no_circulo)
    {
        unsigned int seed = (unsigned int)time(NULL) ^
            (unsigned int)(omp_get_thread_num() + 1);
        long long contador_local = 0;

        #pragma omp for
        for (long long i = 0; i < total_pontos; i++) {
            double x = (double)rand_r(&seed) / (double)RAND_MAX;
            double y = (double)rand_r(&seed) / (double)RAND_MAX;

            if (x * x + y * y <= 1.0) {
                contador_local++;
            }
        }

        #pragma omp atomic
        pontos_no_circulo += contador_local;
    }

    double tempo_total = omp_get_wtime() - t_inicio;
    double pi_estimado = 4.0 * (double)pontos_no_circulo / (double)total_pontos;

    printf("Threads ativas : %d\n", omp_get_max_threads());
    printf("Total de pontos: %lld\n", total_pontos);
    printf("Pontos no circulo : %lld / %lld\n", pontos_no_circulo, total_pontos);
    printf("Pi Estimado       : %.10f\n", pi_estimado);
    printf("Pi Real (PI)      : %.10f\n", PI);
    printf("Erro Absoluto     : %.10e\n", fabs(pi_estimado - PI));
    printf("Tempo de Execucao : %.6f segundos\n", tempo_total);

    return 0;
}
```

Figura 5: Versão com reduction

```
#define PI 3.14159265358979323846

int main(int argc, char **argv) {
    long long total_pontos = 10000000LL;
```

```
    long long pontos_no_circulo = 0;

    double t_inicio = omp_get_wtime();

    #pragma omp parallel shared(total_pontos) reduction(+:pontos_no_circulo)
    {
        unsigned int seed = (unsigned int)time(NULL) ^ (unsigned int)(omp_get_thread_num()
+ 1);

        #pragma omp for
        for (long long i = 0; i < total_pontos; i++) {
            double x = (double)rand_r(&seed) / (double)RAND_MAX;
            double y = (double)rand_r(&seed) / (double)RAND_MAX;

            if (x * x + y * y <= 1.0) {
                pontos_no_circulo++;
            }
        }
    }

    double tempo_total = omp_get_wtime() - t_inicio;
    double pi_estimado = 4.0 * (double)pontos_no_circulo / (double)total_pontos;

    printf("Threads ativas : %d\n", omp_get_max_threads());
    printf("Total de pontos: %lld\n", total_pontos);
    printf("Pontos no circulo : %lld / %lld\n", pontos_no_circulo, total_pontos);
    printf("Pi Estimado       : %.10f\n", pi_estimado);
    printf("Pi Real (PI)      : %.10f\n", PI);
    printf("Erro Absoluto     : %.10e\n", fabs(pi_estimado - PI));
    printf("Tempo de Execucao : %.6f segundos\n", tempo_total);

    return 0;
}
```

5. RESULTADOS

Abaixo, é possível visualizar o resultado de desempenho de compilação dos códigos.

Tabela 1: Comparação de desempenho das estratégias adotadas

Threads	critical + contador local	critical dentro do loop	atomic dentro do loop	atomic + contador local	reduction
2	0,027424	0,283697	0,146788	0,02746	0,027499
4	0,014633	0,537995	0,153960	0,014107	0,014253
6	0,010156	0,653806	0,169313	0,010543	0,010266
8	0,008783	0,766625	0,146079	0,008553	0,008746
10	0,006825	0,896995	0,16113	0,007332	0,00732
12	0,006303	1,086724	0,149855	0,006225	0,006347
14	0,006008	1,276554	0,161927	0,005817	0,005625
16	0,015208	1,459842	0,171483	0,015289	0,015796
18	0,00777	1,680201	0,164824	0,008795	0,007503
20	0,007389	1,711141	0,125566	0,008261	0,007122
22	0,008722	1,668848	0,146525	0,008531	0,009903
24	0,00889	1,713007	0,151689	0,009829	0,008498

Figura 6: Comparação de desempenho das estratégias adotadas em função das threads

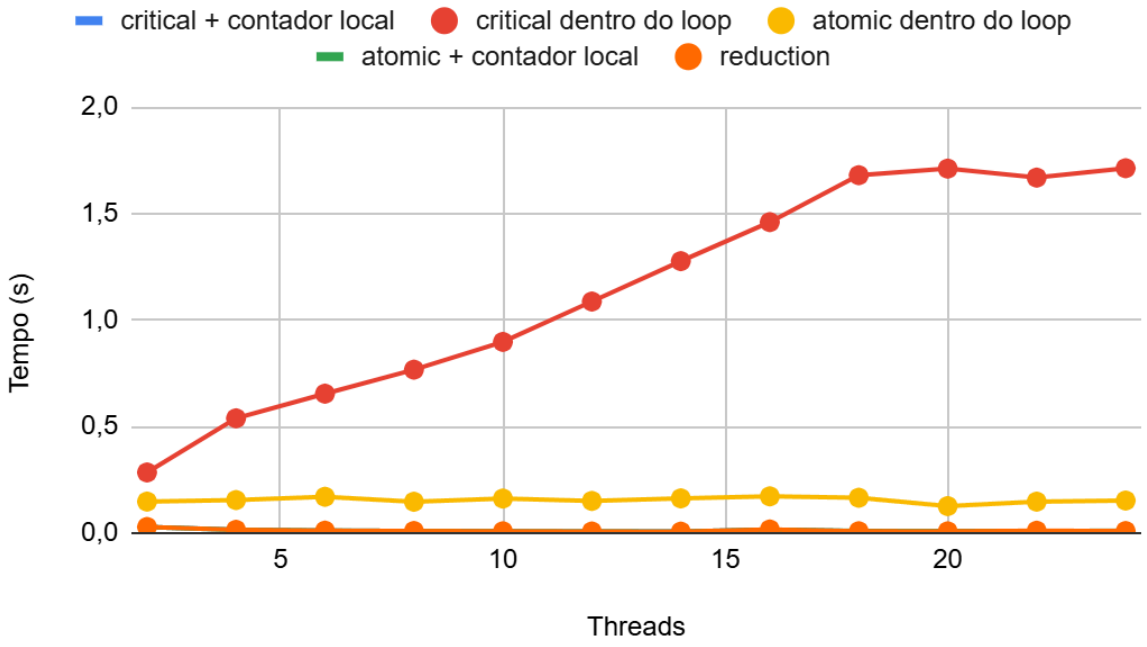
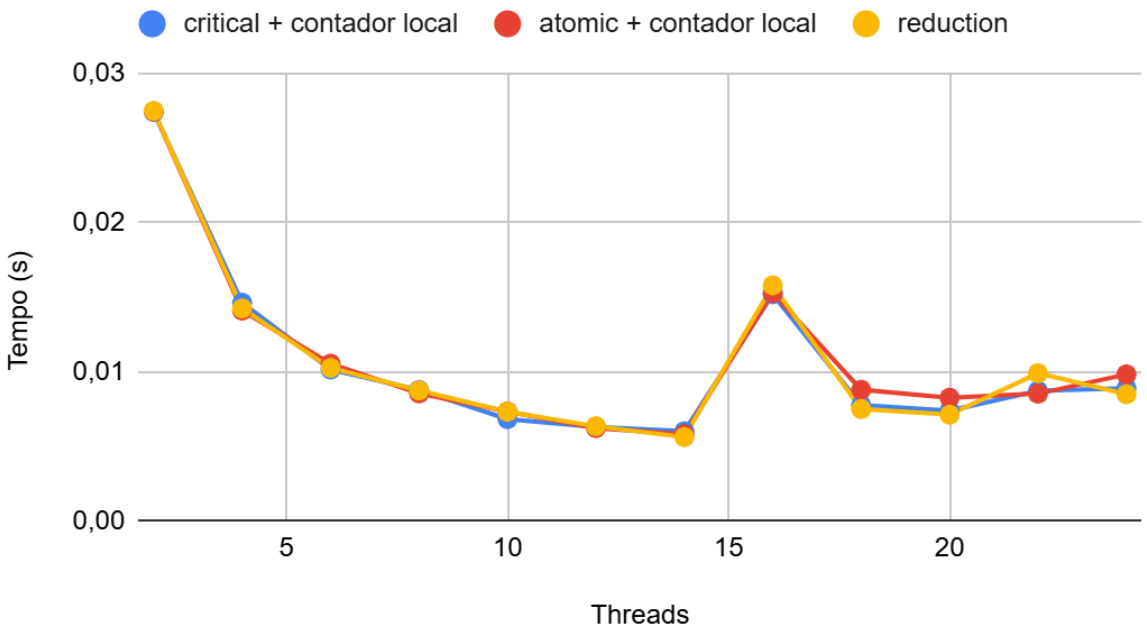


Tabela 7: Comparação de desempenho das estratégias adotadas com contador local em função das threads



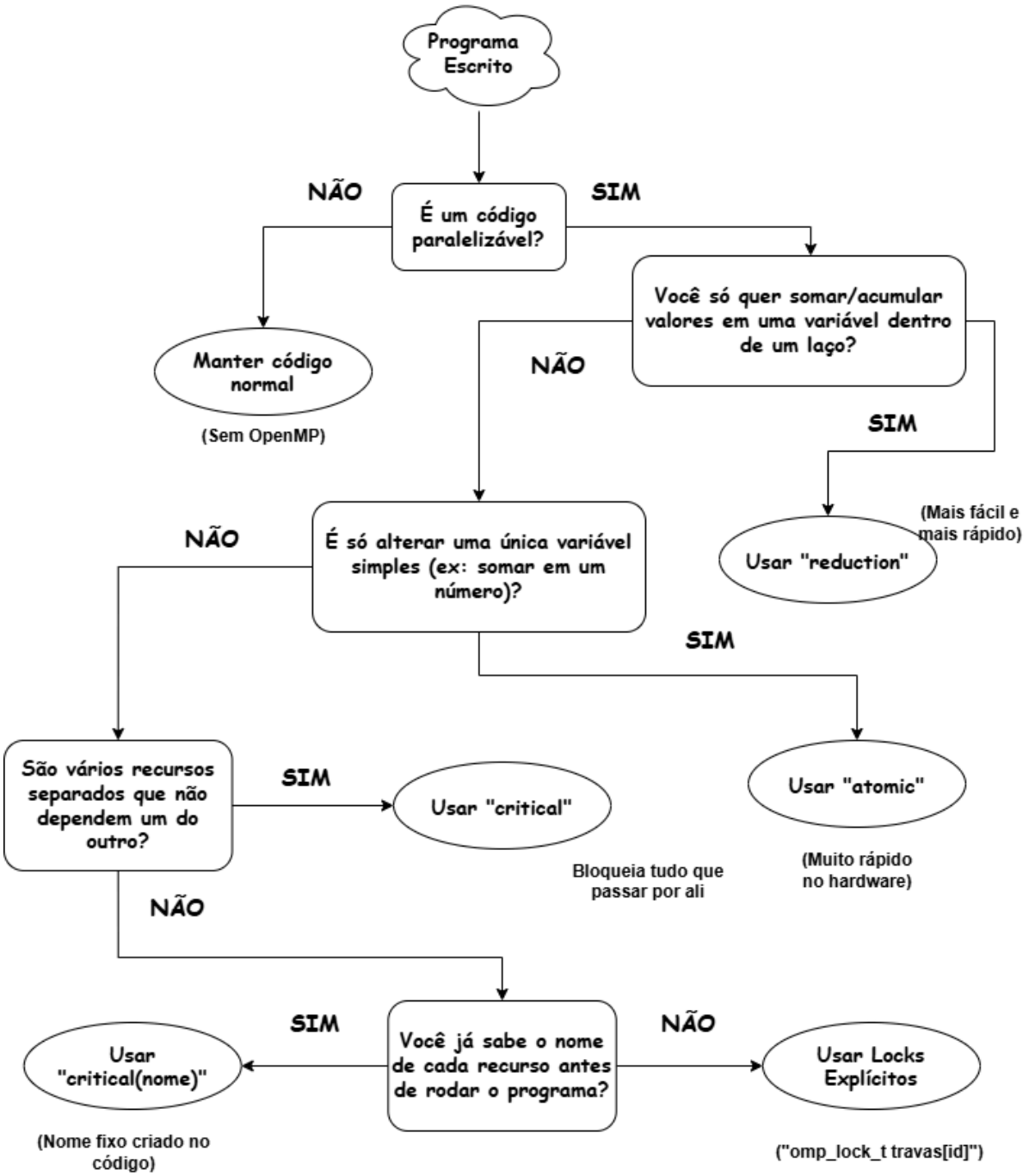
A análise dos dados da Tabela 1 evidencia o impacto direto do mecanismo de sincronização e do nível de contenção de memória no desempenho da aplicação, revelando uma clara divisão entre as abordagens de sincronização interna ao laço e as estratégias baseadas em acúmulo local. O uso do “#pragma omp critical” diretamente dentro do laço principal apresentou o pior desempenho global, sofrendo uma degradação severa com o aumento de “threads” (o tempo saltou de 0,283 segundos com 2 threads para 1,7123 segundos com 24 threads, evidenciando o gargalo causado pela serialização de software).

A aplicação de “#pragma omp atomic” dentro do laço mitigou essa degradação drástica ao utilizar instruções nativas de hardware. Contudo, a contínua disputa pelo mesmo endereço de memória manteve a execução uma ordem de grandeza mais lenta em comparação às soluções otimizadas.

Em contrapartida, as implementações que eliminaram a contenção durante as iterações (“Critical + Contador Local”, “Atomic + Contador Local” e “Reduction”) apresentaram tempos de execução praticamente idênticos e de altíssimo desempenho em todas as configurações. Todas as três variantes escalaram com quase perfeição até 14 threads, exibindo uma oscilação pontual em 16 threads decorrente de “overhead” de gerenciamento do ambiente de execução, para então se estabilizarem. Dessa forma, embora a criação manual de contadores locais atinja

paridade de desempenho, a cláusula “reduction” consolida-se como a solução ideal, pois entrega o tempo mínimo de execução sem adicionar complexidade ao código.

6. DIAGRAMA



7. CONSIDERAÇÕES FINAIS

As considerações finais deste experimento evidenciam que a escolha do mecanismo de sincronização em OpenMP deve equilibrar critérios de desempenho, granularidade do problema e produtividade no desenvolvimento. Para a agregação de dados e reduções escalares em laços independentes (como no estimador de Monte Carlo), a cláusula reduction consolida-se como a escolha ideal. Ela oferece o máximo desempenho ao eliminar a contenção de memória e automatiza o gerenciamento de variáveis privadas, entregando um código limpo, legível e imune a condições de corrida.

As demais diretivas e estratégias avaliadas desempenham papéis fundamentais em cenários estruturais específicos.

- A cláusula **reduction** deve ser a escolha prioritária em operações de agregação matemática (soma, produto, min/max e operadores lógicos) dentro de laços paralelos onde cada iteração é independente e o resultado final precisa ser consolidado em uma única variável compartilhada.
- O #pragma omp **atomic** é recomendado para atualizações pontuais e simples em variáveis escalares de memória compartilhada quando o código não se encaixa na estrutura de um laço de redução, sendo a melhor alternativa para sincronizações no nível de hardware com baixo overhead.
- O #pragma omp **critical (sem nome)** aplica-se à proteção de blocos de código mais amplos ou complexos que envolvem múltiplas instruções, chamadas de função ou estruturas não suportadas por instruções atômicas, devendo ser restrito a trechos de baixa frequência de execução para evitar a serialização do algoritmo.
- Já o #pragma **omp critical(nome)** é ideal no gerenciamento de múltiplos recursos compartilhados independentes, garantindo que threads operando em estruturas distintas não bloqueiem umas às outras.
- Por fim, os **Locks Explícitos (omp_lock_t)** são indispensáveis em arquiteturas complexas com estruturas de dados dinâmicas ou não estruturadas (como grafos, árvores e vetores de listas), onde o controle de concorrência exige bloqueios dinâmicos de granularidade fina (fine-grained) que ultrapassam a sintaxe baseada em blocos estruturados dos pragmas convencionais.